

Phân loại cảm xúc đa nhãn với GoEmotions: So sánh mô hình cổ điển, zero-shot và Transformer

Bảo Duy Nguyễn Quốc Khánh Đức Trí Nhật Huy

Đơn vị:

Email:

Tóm tắt— Nghiên cứu triển khai phân loại cảm xúc đa nhãn trên GoEmotions, gồm 27 cảm xúc và neutral, giữ official split 43.410/5.426/5.427 câu train/validation/test. A sử dụng TF-IDF + One-vs-Rest Logistic Regression; B dùng BART-MNLI zero-shot không fine-tune; C gồm BERT-base-cased, RoBERTa-base và DistilBERT với seed 42, 123, 2026. Nghiên cứu đánh giá Macro/Micro-F1, precision/recall và Hamming Loss; khảo sát class weighting, ngưỡng riêng và F1 năm nhãn hiếm xác định từ train. Trên validation, A standard @0,5 đạt Macro-F1 0.2025; A balanced với ngưỡng riêng đạt 0.4901. Điểm tuned-validation có thể lạc quan vì dùng lại dữ liệu chọn ngưỡng. Bản cập nhật này có 2/9 run C full; những bằng thiếu chưa được dùng xếp hạng. Mọi số lấy từ artifacts thực tế, tách smoke/full và validation/test. Demo được thiết kế dùng C chọn bằng validation. Bài viết diễn giải đầu ra, lỗi và giới hạn domain; liên hệ chuỗi dữ liệu→quyết định→giá trị trong Case Study 4 của Jay Lee, không quy số tiết kiệm của nhà máy thành ROI của mô hình NLP.

Từ khóa— GoEmotions; xử lý ngôn ngữ tự nhiên; phân loại đa nhãn; TF-IDF; Logistic Regression; Transformer; ngưỡng dự đoán.

I. GIỚI THIỆU

Cảm xúc trong văn bản có thể đồng thời thuộc nhiều trạng thái. Một câu cảm ơn có thể thể hiện cả biết ơn và ngưỡng mộ; một phản hồi phản nản có thể chứa cả buồn và khó chịu. Vì vậy, chỉ gán một nhãn tích cực, tiêu cực hoặc trung tính chưa phản ánh đầy đủ bài toán. GoEmotions được Demszyk và cộng sự công bố tại ACL 2020, cung cấp bình luận Reddit tiếng Anh với 27 cảm xúc chi tiết và neutral. Nguồn dữ liệu, taxonomy và baseline BERT của bài báo là nền tảng trực tiếp của đề án [1].

Nhiệm vụ của nghiên cứu là nhận một văn bản x và dự đoán một tập con của 28 nhãn. Đầu ra ban đầu là 28 điểm số; một luật ngưỡng chuyển điểm thành các nhãn được chọn. Bài toán này là multi-label classification, khác với multi-class classification yêu cầu chọn đúng một lớp. Neutral là nhãn thật trong nguồn; không tự động thay tập dự đoán rỗng bằng neutral. Một vector 28 phần tử 0/1 có thể có nhiều số 1, cũng có thể không có nhãn dự đoán vượt ngưỡng.

Đề án so sánh ba hướng: A là TF-IDF kết hợp Logistic Regression theo One-vs-Rest; B là mô hình có sẵn thực hiện zero-shot mà không fine-tune trên GoEmotions; C là ba Transformer được fine-tune: BERT, RoBERTa và DistilBERT. Ba C được đánh giá với các seed 42, 123 và 2026, theo yêu cầu môn học. Demo D dùng checkpoint của C được lựa chọn từ validation. Google Research xác nhận baseline gốc dùng BERT-base-cased; do đó C1 của đề án chọn biến thể cased để có quan hệ rõ với hướng tiếp cận của bài nền tảng [2].

Ba câu hỏi nghiên cứu được đặt ra. Thứ nhất, biểu diễn từ vựng TF-IDF có tạo được một mốc so sánh hữu ích cho cảm xúc chi tiết hay không? Thứ hai, zero-shot và fine-tune thay đổi chất lượng, loại lỗi và nhu cầu dữ liệu như thế nào trong cùng taxonomy? Thứ ba, class weighting và ngưỡng riêng từng nhãn có cải thiện F1 của nhãn hiếm, đồng thời làm thay đổi precision/recall và số nhãn dự đoán ra sao? Bài viết trình bày thiết kế thực nghiệm, số đo có bằng chứng, các phần còn thiếu tại thời điểm xuất và giới hạn ứng dụng; không suy ra hiệu quả kinh doanh từ F1.

II. CÔNG TRÌNH LIÊN QUAN

GoEmotions đóng góp bộ dữ liệu và taxonomy cảm xúc chi tiết, đồng thời khảo sát khả năng học những trạng thái gần nhau và thường đồng xuất hiện [1]. Nghiên cứu này giữ taxonomy, official split và bài toán đa nhãn của nguồn. A cổ điển và B zero-shot là những mốc so sánh bổ sung theo yêu cầu đề án; chúng không được mô tả là baseline BERT của paper. Các bảng giữa hai công trình chỉ so sánh được khi cùng split, loại metric, cách tổng hợp và luật dự đoán. Đặc biệt, độ biến động F1 giữa các nhãn trong một bảng của bài gốc khác với độ lệch chuẩn giữa ba seed của đề án.

BERT học biểu diễn theo ngữ cảnh bằng Transformer hai chiều, sau đó có thể fine-tune cho tác vụ phân loại [3]. Trong đề án, một head tuyến tính đưa biểu diễn câu thành 28 logits; sigmoid và loss nhị phân cho từng nhãn cho phép nhiều cảm xúc cùng tồn tại. C1 sử dụng BERT-base-cased. RoBERTa thay đổi quy trình tiền huấn luyện BERT và là một kiến trúc so sánh C2 [4]. DistilBERT sử dụng knowledge distillation để tạo mô hình nhỏ hơn, phù hợp làm C3 và khảo sát sự đánh đổi giữa tài nguyên và chất lượng [5]. Số tham số hoặc kích thước nhỏ hơn không tự chứng minh tốc độ thực tế trên máy của nhóm; cần đo cùng điều kiện.

Zero-shot dựa trên suy luận ngôn ngữ tự nhiên (natural language inference, NLI) biến tên nhãn thành một giả thuyết và đánh giá mức độ văn bản ủng hộ giả thuyết đó. Yin và cộng sự nghiên cứu cách dùng NLI cho phân loại zero-shot [6]. BART là mô hình encoder-decoder được tiền huấn luyện bằng khôi phục văn bản bị nhiễu [7]. Checkpoint BART-large-MNLI đã được học tác vụ NLI có thể dùng trong pipeline zero-shot; model card hướng dẫn cách dùng candidate labels và hypothesis template [8]. B của đề án không cập nhật trọng số bằng GoEmotions. Nếu hiệu chỉnh ngưỡng trên validation có nhãn, phải gọi đó là hiệu chỉnh quyết định trên validation; không mô tả toàn bộ quá trình là không sử dụng bất kỳ nhãn GoEmotions nào.

Nghiên cứu tập trung kiểm chứng các phương pháp gần trực tiếp với yêu cầu cô giao. Paper nền tảng đúng của đề tài là ACL 2020. Khoảng năm 2022–2026 không được áp vào paper

gốc khi chưa có yêu cầu chính thức bổ sung. Tài liệu phần mềm chính thức giúp xác định cách cài đặt, còn nguồn paper xác định lập luận và hướng mô hình; tài liệu thư viện không thay một bằng chứng thực nghiệm đã chạy trên dữ liệu của nhóm.

III. DỮ LIỆU VÀ PHƯƠNG PHÁP

A. Dữ liệu, mapping và kiểm tra chất lượng

Đồ án dùng cấu hình simplified của GoEmotions với 43.410 mẫu train, 5.426 validation và 5.427 test, tổng cộng 54.263 bình luận. Con số khoảng 58.000 trong mô tả tập gốc không đồng nghĩa với số hàng của simplified official split đang sử dụng. Dataset card và nguồn Google Research được dùng để kiểm nguồn dữ liệu [2], [9]. Revision dữ liệu được khóa là add492243ff905527e67aeb8b80c082af02207c3. Nhân được sắp theo thứ tự canonical trong data/labels.json, từ admiration đến neutral, và dùng một thứ tự này cho tất cả mô hình.

Toàn tập có 63.812 lần gán nhãn, trung bình 1,1760 nhãn mỗi câu; 8.817 câu có ít nhất hai nhãn, chiếm 16,2486%. Trên train, neutral xuất hiện 14.219 lần; grief chỉ có 77 mẫu. Tỷ số support lớn nhất/nhỏ nhất là khoảng 184,66 khi tính cả neutral. Năm nhãn hiếm được xác định trước bằng train là grief, pride, relief, nervousness và embarrassment, có support lần lượt 77, 111, 153, 164 và 303. Xác định nhãn hiếm trên train giúp tránh chọn nhóm cần báo cáo sau khi nhìn test hoặc chỉ chọn nhãn có kết quả tăng.

Mỗi mẫu có ID, văn bản và danh sách nhãn. Vector y có độ dài $L = 28$, $y_j = 1$ khi câu mang nhãn j . Những mẫu neutral đi kèm nhãn khác được giữ nguyên theo annotation thay vì tự sửa taxonomy. Việc đọc dữ liệu kiểm số nhãn, nhãn ngoài phạm vi, ID thiếu/lặp, kích thước split, văn bản rỗng và mapping. ID không trùng giữa các split, nhưng vẫn có exact text trùng: 41 văn bản train/validation, 32 văn bản train/test liên quan 37 hàng test. Benchmark chính giữ official test 5.427 hàng; tập 5.390 hàng không trùng exact text với train chỉ là phân tích bổ sung. Kiểm exact text không bảo đảm loại hết trùng ngữ nghĩa.

Tiền xử lý A chuẩn hóa khoảng trắng và dùng tokenizer của TF-IDF, giữ các từ phủ định, dấu hiệu ngôn ngữ và nội dung cảm xúc. Không mặc định xóa stopwords, stem hoặc lemmatize mọi câu: loại bỏ “not” có thể đảo nghĩa. Mỗi C dùng tokenizer tương ứng checkpoint của chính nó. Từ vựng, IDF và tham số mô hình chỉ được học trên train. Validation phục vụ lựa chọn cấu hình/epoch/ngưỡng; test chỉ dùng đánh giá sau khi các quyết định được khóa. Thông tin nhãn, phân phối và ví dụ test trong EDA được ghi nhận minh bạch; không dùng chúng để chỉnh mô hình hoặc chọn ngưỡng.

B. A: TF-IDF và One-vs-Rest Logistic Regression

Term frequency-inverse document frequency (TF-IDF) biểu diễn văn bản bằng trọng số của từ hoặc cụm từ. Với số văn bản train N , $df(t)$ là số văn bản chứa từ t , IDF có smoothing được định nghĩa như (1). $TF(t,d)$ là số lần t xuất hiện trong văn bản d ở cấu hình của nhóm; sau khi nhân IDF, vector được chuẩn hóa L2 [10].

$$idf(t) = \log((1 + N)/(1 + df(t))) + 1. \quad (1)$$

Từ phổ biến ở gần như mọi văn bản có IDF thấp hơn từ đặc trưng. TF-IDF không tự hiểu một từ là vui hay buồn; ý nghĩa phân loại được học thông qua trọng số Logistic Regression. Unigram giúp tìm từ như “thanks”; bigram biểu diễn cụm như “not happy”. Vì biểu diễn chủ yếu theo từ/cụm từ, A có thể gặp khó khăn với ngữ cảnh dài, mỉa mai và cùng một từ xuất hiện ở các ý nghĩa khác nhau.

Mô hình A dùng word unigram + bigram, min_df = 2, max_features = 100.000; từ vựng thực tế có 58.338 đặc trưng. Logistic Regression có $C = 1$, solver liblinear, max_iter = 1.000, random_state = 42 và L2 regularization [11]. One-vs-Rest huấn luyện 28 bộ phân loại nhị phân, mỗi bộ trả lời câu hỏi “câu này có cảm xúc j không?” [12]. Các bộ phân loại cùng nhìn một vector TF-IDF, nhưng có trọng số và bias riêng.

$$p_j(x) = \sigma(w_j^T x + b_j), \sigma(z) = 1/(1 + \exp(-z)). \quad (2)$$

Mỗi p_j nằm trong $[0,1]$, nhưng tổng của 28 giá trị không cần bằng 1. Đây là lý do dùng sigmoid cho multi-label; softmax và argmax sẽ ép các nhãn cạnh tranh để chỉ giữ một cảm xúc. Quyết định cuối là $\hat{y}_j = 1[p_j \geq \tau_j]$. Ở baseline gốc $\tau_j = 0,5$ cho mọi nhãn. Mô hình không được gọi là không dùng AI chỉ vì nó cổ điển: Logistic Regression vẫn là mô hình học máy, được học từ các câu và nhãn train.

A có hai chế độ trọng số: standard và balanced. Class_weight="balanced" được áp trong mỗi bài toán nhị phân với trọng số $N/(2n_c)$, n_c là số mẫu thuộc lớp c của nhãn đang học. Nhãn hiếm có lớp dương ít nên lớp dương nhận trọng số lớn hơn. Weighting có thể giúp recall nhưng cũng tăng false positives; không mặc định mọi metric sẽ tăng. Nghiên cứu tách các cặp standard→balanced ở cùng ngưỡng và fixed→tuned ở cùng chế độ để nhận diện tác động từng thay đổi, đồng thời báo standard fixed→balanced tuned như cải tiến tổng hợp.

Ví dụ minh họa được tạo để giải thích, không phải kết quả thực nghiệm: câu “Thank you, I am impressed” có các điểm gratitude = 0,72, admiration = 0,48, neutral = 0,12. Với ngưỡng 0,5, mô hình chọn gratitude; khi ngưỡng admiration đã được validation chọn thành 0,45, nó chọn gratitude và admiration. Hạ ngưỡng không thay trọng số LR, mà thay cách quyết định từ cùng điểm số. Những điểm ví dụ này không được đưa vào bảng F1 của GoEmotions.

C. B: BART-MNLI zero-shot

B dùng facebook/bart-large-mnli trong pipeline zero-shot classification. Văn bản là premise; tên mỗi nhãn được đưa vào template “This text expresses {}.” để tạo hypothesis. Với candidate labels là 28 nhãn canonical và multi_label=True, pipeline đánh giá từng nhãn độc lập, cho phép nhiều nhãn được chọn [8]. Điểm số NLI không mặc nhiên được xem là xác suất cảm xúc đã hiệu chuẩn trên GoEmotions. Tên nhãn và template ảnh hưởng kết quả; cả hai được khóa và ghi trong metadata.

Không fine-tune B trên train GoEmotions. Hàng B gốc dùng ngưỡng 0,5; những hàng global/per-label tuning, nếu có, được ghi riêng. Các điểm NLI được lưu theo ID và mapping nhãn để dùng chung module metrics. Lưu riêng smoke/pilot và

full ngăn một lần kiểm vài câu bị hiểu thành benchmark cả tập. Zero-shot giảm nhu cầu huấn luyện bổ sung cho tác vụ, nhưng vẫn có chi phí tải mô hình, chạy nhiều giả thuyết và giới hạn domain/nhân; không đồng nghĩa với một hệ thống miễn phí hoặc không cần kiểm tra.

D. C: ba Transformer fine-tune

C1 dùng google-bert/bert-base-cased, C2 dùng FacebookAI/roberta-base và C3 dùng distilbert/distilbert-base-uncased. Mỗi mô hình dùng đúng tokenizer và kiến trúc tương ứng; từ vựng của BERT không được dùng thay tokenizer RoBERTa. Head phân loại có 28 logits, problem_type="multi_label_classification", label mapping đúng dữ liệu. Sigmoid được áp khi suy luận; lúc huấn luyện dùng BCEWithLogitsLoss trên logits để tránh thực hiện sigmoid hai lần [13].

$$\mathcal{L} = -(1/NL) \sum_j \sum_i [y_{ij} \log \sigma(z_{ij}) + (1-y_{ij}) \log (1-\sigma(z_{ij}))]. \quad (3)$$

Loss trên từng nhân độc lập cho phép một mẫu có nhiều nhân dương. Khi thử C có weighting, positive weight phải được tính từ train cho từng nhân và lưu rõ; không dùng support test. Nhánh standard và weighted cần tách thư mục để không ghi đè kết quả. C weighted là phương án bổ sung; class weighting A và per-label threshold đã là những hướng nâng cao trực tiếp phù hợp yêu cầu đề tài, không bắt buộc tự dựng BERT từ đầu.

C1 có kế hoạch 4 epoch, learning rate 5×10^{-5} ; C2 và C3 là 3 epoch, learning rate 2×10^{-5} . Mặc định batch size 16, gradient accumulation 1, max_length 128, dynamic padding theo batch, AdamW với weight_decay 0,01, warmup 0,1 và gradient clipping 1,0. Cấu hình thực tế của mỗi run nằm trong metadata. Các lựa chọn learning rate/epoch khác nhau là một giới hạn khi quy nguyên nhân chênh lệch hoàn toàn cho kiến trúc; đây là so sánh các hệ thống được triển khai trong đồ án, chưa phải ablation cô lập mọi yếu tố.

Trong từng seed, chọn checkpoint có Macro-F1 validation @0,5 cao nhất, hòa chọn epoch sớm hơn. Kiến trúc dùng cho D được chọn theo mean Macro-F1 validation @0,5 của đủ ba seed ở cả ba C. Checkpoint đại diện của kiến trúc được chọn tiếp bằng validation theo luật đã lưu; không lấy seed có test cao nhất. Ngưỡng, revision model, hash và đường dẫn run được khóa trong hồ sơ. Đầu ra demo phải thuộc đúng checkpoint/mapping/ngưỡng đã chọn.

E. Ngưỡng và quy trình dùng chung

Mỗi hệ thống được đánh giá với fixed 0,5, global threshold và per-label threshold. Grid ứng viên là 0,05 đến 0,95, bước 0,05. Global threshold tối ưu Macro-F1 validation; mỗi ngưỡng riêng tối ưu F1 của một nhân trên validation. Nếu hòa, ưu tiên gần 0,5 rồi chọn giá trị lớn hơn. Việc chọn ngưỡng làm thay đổi precision/recall, vì vậy cần dùng dữ liệu khác train và không tuning bằng test [14].

Cùng validation được dùng chọn ngưỡng rồi tính bảng tuned-validation nên điểm này có thể lạc quan. Kết luận tổng quát cần bảng test sau khi ngưỡng đã cố định. Không chỉnh ngưỡng tiếp để làm test đẹp hơn. Khi không có nhân vượt ngưỡng, demo thông báo chưa có nhân đủ điểm; có thể hiển thị top scores để người dùng hiểu, nhưng top scores không tự biến thành nhân được dự đoán. Không thêm fallback

neutral/argmax vào một hệ thống rồi so sánh với hệ thống khác không có fallback.

IV. THIẾT KẾ THỰC NGHIỆM VÀ KẾT QUẢ

A. Metrics và mức chứng cứ

Với từng nhân j , TP là số dương đoán đúng, FP là nhân đoán thừa, FN là nhân thật bị bỏ sót. Precision $P_j = TP_j / (TP_j + FP_j)$, recall $R_j = TP_j / (TP_j + FN_j)$, $F1_j = 2TP_j / (2TP_j + FP_j + FN_j)$. Mẫu số bằng 0 dùng zero_division=0 trong cùng module đánh giá cho A/B/C [15]. Support là số câu có nhân thật, khác số lần model chọn nhân.

$$\text{MacroF1} = (1/L) \sum_j F1_j. \quad (4)$$

$$\text{MicroF1} = 2 \sum_j TP_j / (2 \sum_j TP_j + \sum_j FP_j + \sum_j FN_j). \quad (5)$$

Macro-F1 cho mỗi nhân cùng trọng lượng, do đó nhạy hơn với chất lượng nhân hiếm. Micro-F1 gộp mọi cặp mẫu/nhân, thường chịu ảnh hưởng lớn từ nhân xuất hiện nhiều. Macro precision/recall cũng là trung bình theo nhân; micro precision/recall tính trên các tổng TP/FP/FN. Hamming Loss là tỷ lệ sai trên $N \times 28$ quyết định nhị phân [16]. Hamming thấp vẫn có thể đi cùng bỏ sót nhiều cảm xúc hiếm vì phần lớn các ô là nhân âm; do vậy không sử dụng nó một mình để chọn hệ thống.

$$\text{HammingLoss} = (1/NL) \sum_j \sum_i 1[y_{ij} \neq \hat{y}_{ij}]. \quad (6)$$

Với $K = 3$ seed, báo trung bình $m = \sum_i s_i / K$ và sample standard deviation theo (7), ddof=1 [17]. Đây là độ biến động giữa các lần huấn luyện, không phải độ biến động giữa 28 nhân, không phải confidence interval và không tự cho một kiểm định ý nghĩa thống kê. A/B một run không nhận std=0 để giả vờ ổn định như C.

$$\text{std} = \sqrt{\sum_i (s_i - m)^2 / (K - 1)}. \quad (7)$$

Python, NumPy, PyTorch CPU/CUDA đều được gieo seed. PyTorch nêu rõ khả năng tái hiện có thể khác giữa phiên bản, nền tảng hoặc phép toán [18]. Metadata lưu seed, phiên bản, device, revision, configuration, epoch được chọn, timestamps và hash các artifact. Environment đã có C1 full: Python 3.13.9, PyTorch 2.13.0+cu130, Transformers 4.57.6, CUDA 13 và NVIDIA GeForce RTX 5060 Laptop GPU, AMP FP16. Thời gian elapsed là toàn run, không chỉ optimizer. Máy đã ngủ ba lần trong quá trình C1 seed42 ngày 08/10/2026 nên không dùng elapsed hiện tại để kết luận kiến trúc nào nhanh hơn.

B. Bảng thực nghiệm theo trạng thái thực tế

Trạng thái 08/10/2026 14:09 (UTC+7): Bản cập nhật chưa đủ benchmark cuối. Có 8/72 hàng kết quả và 6/36 nhóm tổng hợp theo thiết kế; C full 2/9, B full chưa có. Số hàng phụ thuộc artifact đã hoàn thành, không tính smoke. Dấu — là thiếu/chưa đủ ba seed hoặc không áp dụng, không phải F1=0. A-S là A standard; A-W là A balanced; C1/C2/C3 là BERT/RoBERTa/DistilBERT. Val là validation, N=5.426; Test N=5.427.

BẢNG I. SO SÁNH HỆ THỐNG GỐC, NGƯỠNG 0,5.

Hệ	Run	Val Macro	Test Macro	Test Micro	Test H
A-S	1	0.2025	—	—	—
A-W	1	0.4562	—	—	—

Hệ	Run	Val Macro	Test Macro	Test Micro	Test H
B	—	—	—	—	—
C1	—	—	—	—	—
C2	—	—	—	—	—
C3	—	—	—	—	—

Cần đủ ba seed để có hàng C mean $\pm$ std. Một C đã hoàn tất vẫn xuất hiện trong bảng từng seed ở dưới; không điền điểm đó vào hàng trung bình ba seed.

BẢNG II. ABLATION A TRÊN VALIDATION.

Cấu hình	Macro	Micro	P μ	R μ	H
A-S_fixed	0.2025	0.3760	0.7254	0.2538	0.0354
A-S_global	0.4094	0.5100	0.4102	0.6741	0.0544
A-S_tuned	0.4391	0.5427	0.4876	0.6119	0.0433
A-W_fixed	0.4562	0.5099	0.4158	0.6592	0.0532
A-W_global	0.4660	0.5176	0.4524	0.6047	0.0473
A-W_tuned	0.4901	0.5467	0.4796	0.6356	0.0443

fixed: ngưỡng 0,5; global: một ngưỡng chọn trên val; tuned: riêng từng nhân chọn trên val. Các số là full validation, nhưng global/tuned không phải ước lượng độc lập với bước calibration.

Bảng test ablation chưa có đủ artifact. Chưa thể kết luận cải thiện tổng quát trên test từ bảng tuned-validation.

BẢNG IV. PRECISION/RECALL CÙNG NGUỖNG 0,5.

Hệ	Split	P macro	R macro	P micro	R micro
A-S	Val	0.5571	0.1436	0.7254	0.2538
A-W	Val	0.3858	0.5801	0.4158	0.6592

Split được ghi theo từng hàng. Không xếp hạng một hàng val với một hàng test như cùng phép đo.

BẢNG V. TỪNG SEED C; MACRO-F1.

C	Seed	Epoch	Val fixed	Test fixed	Test tuned
C1	42	4	0.4641	—	—
C1	123	4	0.4773	—	—
C1	2026	—	—	—	—
C2	42	—	—	—	—
C2	123	—	—	—	—
C2	2026	—	—	—	—
C3	42	—	—	—	—
C3	123	—	—	—	—
C3	2026	—	—	—	—

Epoch chọn bằng validation @0,5 của chính seed. Bảy metrics từng seed/split/ngưỡng nằm trong reports/project_results/all_runs.csv; mean_std.csv giữ toàn bộ nhóm, không chỉ bảng rút gọn.

B pilot, ngoài bảng benchmark: mới đo 8 câu, Macro-F1 0.0721, Micro-F1 0.0781, Hamming 0.5268. Số này chỉ kiểm pipeline; không dùng để kết luận B tốt/kém hơn các hàng full.

Các phần chưa đủ trong summary: A test; B full validation + frozen protocol; C bert seed 42 frozen thresholds; C bert seed 42 test; C bert seed 123 frozen thresholds; C bert seed 123 test; C bert seed 2026 full; C roberta seed 42 full; C

roberta seed 123 full; C roberta seed 2026 full; C distilbert seed 42 full; C distilbert seed 123 full; C distilbert seed 2026 full; D selected_model from 3 architectures $\times$ 3 seeds.

Hồ sơ đối chiếu gồm summary.json (revision, sources và hashes), all_runs.csv, mean_std.csv, per_label.csv, final_protocol.json của từng run và reports/reproducibility. SHA-256 summary của bản xuất này lưu trong BAI_BAO_GOEMOTIONS_IEEE_KIEM_CHUNG.json.

Trọng số lớn/checkpoint lưu riêng trong data/processed, không coi việc vắng chúng trên Git là thiếu mô tả phương pháp.

C. Nhân hiếm, weighting và threshold tuning

BẢNG VI. NĂM NHÂN HIẾM TRƯỚC/SAU — VALIDATION CÓ CALIBRATION.

Nhân	Hệ	Train+	Eval+	F1 trước	F1 sau	Δ
grief	A	77	13	0.0000	0.4375	+0.4375
grief	C1	77	—	—	—	—
grief	C3	77	—	—	—	—
grief	C2	77	—	—	—	—
pride	A	111	15	0.0000	0.6087	+0.6087
pride	C1	111	—	—	—	—
pride	C3	111	—	—	—	—
pride	C2	111	—	—	—	—
relief	A	153	18	0.0000	0.1739	+0.1739
relief	C1	153	—	—	—	—
relief	C3	153	—	—	—	—
relief	C2	153	—	—	—	—
nervousness	A	164	21	0.0000	0.3125	+0.3125
nervousness	C1	164	—	—	—	—
nervousness	C3	164	—	—	—	—
nervousness	C2	164	—	—	—	—
embarrassment	A	303	35	0.1081	0.5507	+0.4426
embarrassment	C1	303	—	—	—	—
embarrassment	C3	303	—	—	—	—
embarrassment	C2	303	—	—	—	—

Train+/Eval+ là support dương trong train/split đánh giá. A: standard fixed $\rightarrow$ balanced tuned, một run; C: fixed $\rightarrow$ tuned cùng checkpoint/seed, đủ ba seed mới có mean $\pm$ std. Δ là trung bình chênh lệch cặp seed; mọi tăng/giảm và std của Δ được giữ trong rare_before_after.csv. Ô — là thiếu/chưa đủ dữ liệu.

Bảng hiện tại dùng validation đã hiệu chỉnh ngưỡng; test trước/sau chưa đủ. Không gọi các mức tăng này là cải thiện test hay chứng cứ triển khai thực tế.

Khi đối chiếu standard fixed với balanced tuned của A, cả class weighting và luật ngưỡng đều thay đổi. Mức tăng tổng hợp không được quy toàn bộ cho một yếu tố. Hàng standard tuned và balanced fixed giúp đọc riêng từng tác động. Với C, so sánh fixed $\rightarrow$ tuned trên cùng seed/checkpoint rồi báo

mean±std giữa ba seed. Báo đủ mọi nhân đã xác định từ train, kê cả nhân giảm, và giữ support để diễn giải biến động. Việc không tăng F1 ở một nhân hiểm vẫn là kết quả đáng báo cáo.

D. Phân tích lỗi có ví dụ

Ví dụ A standard @0,5 trên validation đã kiểm. ID eczwil0, “I am so proud of this community.”, nhân thật pride nhưng dự đoán rằng, score pride=0,3776; ID ed832y6, “Homeopaths love it!”, nhân thật neutral nhưng dự đoán love, score love≈1; ID eczdvn, “Thank you. I really appreciate your response”, nhân thật admiration/gratitude nhưng chỉ tìm gratitude, score admiration=0,4989. Đây là lỗi A; không gán các ví dụ này thành lỗi của C khi chưa đọc scores C.

C1/C2/C3 chưa có hồ sơ so sánh lỗi test đầy đủ. Cần ba checkpoint đại diện chọn bằng validation, cùng ID và ít nhất ba nhóm. Có code phân tích chưa chứng minh đã hoàn tất số đo.

Ba nhóm lỗi đối chiếu C1/C2/C3 là bỏ sót nhân hiểm (rare false negative), đoán đúng một phần nhưng thiếu nhân khác trong câu đa nhân (partial_multi_label), và đồng thời bỏ sót nhân thật/thêm nhân sai (missed_extra_pair). Ví dụ A còn minh họa false positive. Một mẫu có thể nằm trong nhiều nhóm; không cộng số lượng nhóm như các tập rời nhau. Các cặp “nhân thật bị bỏ sót → nhân dự đoán thừa” trên cùng câu hỗ trợ tìm cảm xúc dễ nhầm, nhưng không thay một confusion matrix đơn nhân. Ví dụ ngữ nghĩa phải được đọc thủ công trước khi gọi là mỉa mai, phủ định hay slang; chỉ từ score không suy ra chắc chắn nguyên nhân.

V. THẢO LUẬN, DEMO VÀ GIÁ TRỊ ỨNG DỤNG

A. Diễn giải kết quả và demo C

Chưa đủ ba seed của cả ba C nên chưa xếp hạng hoặc khẳng định C1/C2/C3 tốt nhất/ổn định nhất. Một run full giúp kiểm triển khai, nhưng không thay điều kiện so sánh mean±std. A-W tuned-validation không được so với một C fixed-validation rồi quy ưu thế hoàn toàn cho mô hình.

Chưa có selected_model.json hợp lệ cho best C ở thời điểm xuất; chưa xác nhận demo đạt yêu cầu D.

Chưa có hồ sơ verify_demo hoàn tất tại thời điểm xuất. Giao diện cần kiểm với checkpoint thật.

Demo sử dụng Gradio để nhập câu tiếng Anh, hiển thị nhân vượt ngưỡng, điểm số và các trạng thái dự đoán [19]. Giao diện không giả lập kết quả của một model chưa có checkpoint. Hồ sơ selected_model.json, mapping nhân, threshold và hash checkpoint phải khớp. Kiểm demo gồm câu thường, câu đa cảm xúc, dữ liệu rỗng và tình huống không nhân vượt ngưỡng. Smoke hoặc ảnh chụp một lần không bảo đảm máy khác vẫn chạy; cần hướng dẫn môi trường, kiểm import/khởi động và thời điểm ghi nhận bằng chứng.

Đầu ra hỗ trợ con người đọc và sắp xếp phản hồi theo cảm xúc chi tiết. Ví dụ trong một hàng đợi phản hồi khách hàng tiếng Anh, nhân annoyance/disappointment có thể giúp ưu tiên nhân viên đọc các câu cần xem xét, gratitude hỗ trợ tìm phản hồi tích cực, và grief/nervousness có thể gợi ý kiểm tra thủ công. Những ví dụ này là đề xuất ứng dụng, chưa phải một hệ thống vận hành doanh nghiệp. Không coi nhân cảm xúc là chẩn đoán tâm lý hoặc một quyết định tự động về con người.

B. Liên hệ Case Study 4 của Lee

Case Study 4, mục 4.2.3.1, trang in 82–88 trong sách Industrial AI của Jay Lee mô tả tối ưu năng lượng tại nhà máy LCD ở Shenzhen. Hệ thống truyền thống dựa vào FMCS/SCADA, ngưỡng báo động, quan sát và kinh nghiệm vận hành. Hướng Industrial AI kết hợp mô hình cơ chế, dự báo nhu cầu, tối ưu vận hành và theo dõi sức khỏe máy nén khí/chiller [20]. Case này dùng dữ liệu thiết bị và năng lượng, không sử dụng GoEmotions hoặc phân loại cảm xúc NLP.

Điểm liên hệ là chuỗi dữ liệu → ước lượng/dự đoán → hành động → giá trị đo được. Trong Case 4, dự báo nhu cầu hỗ trợ vận hành thiết bị phù hợp để tiết kiệm. Trong kịch bản NLP, điểm cảm xúc có thể hỗ trợ nhân viên ưu tiên đọc hoặc tổng hợp xu hướng phản hồi; hành động cuối cùng cần quy tắc vận hành và kiểm tra con người. Nếu không dùng mô hình NLP, nhân viên đọc thủ công hoặc dùng luật từ khóa. Nếu có mô hình, văn bản đi qua biểu diễn/tokenizer, bộ phân loại, ngưỡng rời giao diện; con người xem lại những câu score gần ngưỡng hoặc cảm xúc nhạy cảm. TF-IDF + LR đã là một phương án có mô hình; luật từ khóa cố định là phương án chưa học tham số từ dữ liệu.

Sách báo cáo tính toán tiết kiệm lịch sử năm 2018 trên 10 máy nén khí hơn 300.000 USD/năm và trên 12 chillers hơn 70.000 USD/năm [20]. Đây là số sách cung cấp, chưa được nhóm kiểm toán độc lập, không phải lợi nhuận của mô hình GoEmotions. Đối với NLP, đo giá trị ứng dụng cần thời gian xử lý phản hồi, độ chính xác ưu tiên, tỷ lệ bỏ sót vấn đề quan trọng, số false alarms, chi phí nhân công/hạ tầng và kết quả xử lý khách hàng. Macro-F1 tăng chỉ phản ánh metric trên dataset; chưa tự chứng minh ROI hoặc mức tiết kiệm.

C. Giới hạn và hướng tiếp tục

Reddit tiếng Anh khác phản hồi dịch vụ, văn bản chuyên ngành và tiếng Việt. Taxonomy của GoEmotions có thể chưa trùng nhu cầu doanh nghiệp. Trước triển khai cần chọn domain, tạo dữ liệu gán nhãn tại domain đó, kiểm quy trình gán nhãn, đo model và calibration, xác định khi nào chuyển sang người đọc. Dịch câu tiếng Việt sang tiếng Anh trước suy luận là một hệ thống khác, có thêm lỗi dịch và cần đánh giá riêng; không mặc định mô hình hiện có hiểu tiếng Việt.

Nhân hiểm có ít ví dụ nên thay đổi vài TP/FP/FN có thể làm F1 biến động nhiều. Annotation cảm xúc còn chịu ảnh hưởng diễn giải chủ quan, câu ngắn thiếu ngữ cảnh, slang và nội dung mỉa mai. Official split có exact text overlap đã nêu; cần giữ giới hạn này bên cạnh kết quả. Max_length 128 có thể cắt văn bản dài, nhưng số bị cắt cân đo bằng đúng tokenizer của từng C, không gán thống kê tokenizer uncased cũ cho BERT cased. Ba seed hỗ trợ mô tả ổn định bước đầu; chưa đủ để kết luận ưu thế có ý nghĩa thống kê trong mọi môi trường.

Một mở rộng phù hợp là nghiên cứu calibration, phụ thuộc giữa nhân, dữ liệu domain mới và representation learning. Trước khi thêm kỹ thuật, cần hoàn thành các bảng, đọc lỗi và chốt protocol hiện tại. Mỗi thay đổi mới phải có baseline đối chiếu, đủ số đo và ghi rõ câu hình; không thêm các phương pháp chỉ trong mô tả mà thiếu code hoặc thử nghiệm. Phiên bản này ưu tiên minh bạch số liệu và giải thích được từng bước của mô hình cổ điển cùng các kiến trúc Transformer.

VI. KẾT LUẬN

Đồ án giữ bài toán 28 nhãn và official split của GoEmotions, xây A để giải thích, B không fine-tune và ba C fine-tune, cùng module đánh giá/threshold. Class weighting và ngưỡng riêng được đối chiếu bằng ablation, support nhãn hiếm và các ví dụ lỗi. Hiện C hoàn tất 2/9 run full; phần chưa có B/C/test/lỗi/demo được đánh dấu trong bảng. Không dùng pilot hoặc điểm tuned-validation để thay kết luận test. Cần hoàn tất các artifact thiếu, khóa protocol rồi đánh giá test và kiểm demo trước khi dùng bản nộp cuối. Giá trị đầu ra là cung cấp điểm và nhãn hỗ trợ đọc/phân tích phản hồi. Hiệu quả tại domain doanh nghiệp, tiếng Việt và ROI cần dữ liệu cùng quy trình đo riêng. Liên hệ Case Study 4 giúp giải thích mối quan hệ dữ liệu→quyết định→giá trị, không biến ví dụ tiết kiệm năng lượng thành bằng chứng tài chính của NLP.

PHÂN CÔNG VÀ GHI NHẬN HỖ TRỢ

Phân công xác nhận gồm bốn thành viên: Bảo Duy Nguyễn phụ trách A, điều phối B và data/metrics dùng chung; Quốc Khánh phụ trách C1 BERT và phần đầu/tổng hợp báo cáo; Đức Trí phụ trách C2 RoBERTa và hỗ trợ/bản giao B; Nhật Huy phụ trách C3 DistilBERT và demo từ best C. Đây là trách nhiệm được giao; việc có artifact được công cụ tạo không tự xác nhận từng người đã tự thực hiện hoặc đã bảo vệ được nội dung. Tỷ lệ công sức, MSSV và thông tin hành chính để trống chờ nhóm xác nhận.

Mã nguồn dùng scikit-learn, PyTorch, Hugging Face và Gradio. Công cụ hỗ trợ AI được sử dụng để hỗ trợ soạn code, tra nguồn và biên soạn tài liệu. Nhóm cân đọc, kiểm chạy, đối chiếu số liệu và chịu trách nhiệm về bản nộp. Bản Word/PDF là bản đồ án theo định dạng IEEE conference hai cột; không tự nhận là bài đã được IEEE chấp nhận hoặc xuất bản.

TÀI LIỆU THAM KHẢO

- [1] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, "GoEmotions: A Dataset of Fine-Grained Emotions," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics, 2020, pp. 4040–4054, doi: 10.18653/v1/2020.acl-main.372. <https://aclanthology.org/2020.acl-main.372/>
- [2] Google Research, "GoEmotions: README and official data," Google Research GitHub repository. Accessed: Oct. 8, 2026. [Online]. Available: <https://github.com/google-research/google-research/blob/master/goemotions/README.md>
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, vol. 1, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423. <https://aclanthology.org/N19-1423/>
- [4] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," 2019, arXiv:1907.11692, doi: 10.48550/arXiv.1907.11692. <https://arxiv.org/abs/1907.11692>
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019, arXiv:1910.01108, doi: 10.48550/arXiv.1910.01108. <https://arxiv.org/abs/1910.01108>
- [6] W. Yin, J. Hay, and D. Roth, "Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach," in Proc. EMNLP-IJCNLP, 2019, pp. 3914–3923, doi: 10.18653/v1/D19-1404. <https://aclanthology.org/D19-1404/>
- [7] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in Proc. 58th Annu. Meeting Assoc. Comput. Linguistics, 2020, pp. 7871–7880, doi: 10.18653/v1/2020.acl-main.703. <https://aclanthology.org/2020.acl-main.703/>
- [8] Facebook AI, "bart-large-mnli model card," Hugging Face. Accessed: Oct. 8, 2026. [Online]. Available: <https://huggingface.co/facebook/bart-large-mnli>
- [9] Google Research Datasets, "go_emotions dataset card," Hugging Face. Accessed: Oct. 8, 2026. [Online]. Available: https://huggingface.co/datasets/google-research-datasets/go_emotions
- [10] scikit-learn developers, "TfidfVectorizer," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://scikit-learn.org/1.7/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [11] scikit-learn developers, "LogisticRegression," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://scikit-learn.org/1.7/modules/generated/sklearn.linear_model.LogisticRegression.html
- [12] scikit-learn developers, "OneVsRestClassifier," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: <https://scikit-learn.org/1.7/modules/generated/sklearn.multiclass.OneVsRestClassifier.html>
- [13] PyTorch contributors, "BCEWithLogitsLoss," PyTorch 2.14 documentation. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.pytorch.org/docs/2.14/generated/torch.nn.BCEWithLogitsLoss.html>
- [14] scikit-learn developers, "Tuning the decision threshold for class prediction," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://scikit-learn.org/1.7/modules/classification_threshold.html
- [15] scikit-learn developers, "precision_recall_fscore_support," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://scikit-learn.org/1.7/modules/generated/sklearn.metrics.precision_recall_fscore_support.html
- [16] scikit-learn developers, "hamming_loss," scikit-learn 1.7.2 documentation. Accessed: Oct. 8, 2026. [Online]. Available: https://scikit-learn.org/1.7/modules/generated/sklearn.metrics.hamming_loss.html
- [17] NumPy developers, "numpy.std," NumPy documentation. Accessed: Oct. 8, 2026. [Online]. Available: <https://numpy.org/doc/stable/reference/generated/numpy.std.html>
- [18] PyTorch contributors, "Reproducibility," PyTorch 2.14 documentation. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.pytorch.org/docs/2.14/notes/randomness.html>
- [19] Gradio, "Quickstart," Gradio documentation. Accessed: Oct. 8, 2026. [Online]. Available: <https://www.gradio.app/guides/quickstart>
- [20] J. Lee, Industrial AI: Applications with Sustainable Performance, 1st ed. Singapore: Springer, 2020, ch. 4, sec. 4.2.3.1, pp. 82–88, doi: 10.1007/978-981-15-2144-7. <https://link.springer.com/book/10.1007/978-981-15-2144-7>